Introduction to Databases

SQL & MongoDB with Python

Introduction to Software Systems | Spring ‘26

Why Databases?

Without a Database

- Data in JSON/CSV files
- No concurrent access safety
- No query language — manual parsing
- No data validation or constraints
- Doesn't scale beyond small projects

With a Database

- Structured, organized storage
- Safe concurrent reads & writes
- Powerful query languages
- Data types, constraints, validation
- Scales to millions of records

Database Landscape

Relational (SQL)

MySQL

PostgreSQL

SQLite

Document (NoSQL)

MongoDB

CouchDB

Firestore

Key-Value

Redis

DynamoDB

Memcached

Graph

Neo4j

ArangoDB

Amazon Neptune

Today we focus on: **SQLite (Relational) & MongoDB (Document)**

ACID Properties

Atomicity

Either all operations within the transaction succeed, or none of them do.
If one part fails, the entire transaction rolls back.

Consistency

Guarantees that the database is never in an invalid state.

Isolation

Ensures that concurrent transactions do not interfere with each other.

Durability

Guarantees that once a transaction is committed, its effects are permanent, even in the case of a system crash or power failure.

These are some things a good database system should have.

SQL — The Most Used Database Language

Who uses SQL?

Instagram (PostgreSQL), Wikipedia (MariaDB/MySQL), Banks & Financial systems, Government databases, your college ERP system, almost every web app backend

Popular SQL Variants

PostgreSQL	MySQL	SQLite	SQL Server
Most feature-rich, open source	Most popular, web apps	Serverless, embedded, simple	Microsoft, enterprise

Why SQLite for This Lab?

Zero Setup
No server to install. Comes built into Python's standard library.

Single File
Your entire database is one .db file. Easy to share, backup, delete.

Same SQL Syntax
95% of what you write in SQLite works in PostgreSQL & MySQL.

Used Everywhere
Android, iOS, browsers, apps — SQLite is the most deployed DB.

CRUD Operations

Huh?

These are the 4 most common things you can do using any database system.

But what does it stand for?

CREATE	READ	UPDATE	DELETE

SQL Syntax — SELECT

```
SELECT column1, column2 FROM table_name WHERE condition;
```

Examples from a shop database:

`SELECT * FROM products;`	*Get all columns from products*
`SELECT name, price FROM products;`	*Get only name and price*
`SELECT * FROM products WHERE price > 100;`	*Filter: price greater than 100*
`SELECT * FROM products ORDER BY price DESC;`	*Sort by price, highest first*
`SELECT * FROM products LIMIT 5;`	*Get only first 5 rows*
`SELECT * FROM products WHERE category = 'Electronics' AND price < 500;`	*Multiple conditions with AND*

SQL — CREATE TABLE & INSERT

```sql
CREATE TABLE
 products (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT NOT NULL,
    price    REAL NOT NULL,
    category TEXT,
    stock    INTEGER DEFAULT 0
 );
```

```sql
INSERT INTO
 products (name, price, category, stock)
VALUES ('Wireless Mouse', 299.99, 'Electronics', 50);
```

PRIMARY KEY = unique ID | NOT NULL = required | DEFAULT = fallback value | AUTOINCREMENT = auto-assign ID

SQL — UPDATE & DELETE

UPDATE

```
UPDATE products SET price = 249.99, stock = 45
WHERE name = 'Wireless Mouse';
```

DELETE

```
DELETE FROM products WHERE id = 3;
```

Always use WHERE with UPDATE and DELETE!

Without WHERE, UPDATE changes ALL rows. DELETE without WHERE deletes EVERYTHING.

SQL — JOINs

Combine data from multiple tables

products		
id	name	price
1	Mouse	299
2	Keyboard	599

orders		
order_id	product_id	qty
101	1	2
102	2	1

```sql
SELECT products.name, orders.qty, products.price * orders.qty AS total
FROM orders INNER JOIN products ON orders.product_id = products.id;
```

INNER JOIN returns only rows where there's a match in both tables

NoSQL & MongoDB

A different way to think about data

What is NoSQL?

"Not Only SQL" — databases that don't use the traditional table-based relational model

A MongoDB Document:

```
{
  "name": "Wireless Mouse",
  "price": 299.99,
  "category": "Electronics",
  "specs": {
    "color": "black",
    "wireless": true
  },
  "tags": ["gaming", "bluetooth"]
}
```

Documents, not rows
Data stored as JSON-like objects (BSON)

Flexible schema
Each document can have different fields

Nested data
Objects and arrays inside documents

No JOINs needed
Related data embedded in one document

SQL vs NoSQL — Key Differences

Aspect	SQL (SQLite)	NoSQL (MongoDB)
Data Model	Tables with rows & columns	Collections of JSON documents
Schema	Fixed — defined upfront	Flexible — per document
Relationships	JOINs across tables	Embedded/nested documents
Query Language	SQL (standardized)	MongoDB Query Language
Scaling	Vertical (bigger server)	Horizontal (more servers)
Best For	Structured, relational data	Flexible, evolving data

When to Use Which?

Choose SQL When...

- Data has clear relationships
- Schema is well-defined and stable
- You need complex queries & JOINs
- Data integrity is critical (banking)
- ACID transactions matter

Choose MongoDB When...

- Schema evolves frequently
- Data is nested/hierarchical
- Rapid prototyping & MVPs
- Handling varied data shapes
- Horizontal scaling needed

MongoDB — Connect & Insert

MongoDB (Python)

```python
client = MongoClient(URI)
db = client["shop"]
products = db["products"]

products.insert_one({
  "name": "Wireless Mouse",
  "price": 299.99,
  "category": "Electronics",
  "stock": 50
})
```

SQL Equivalent

```sql
CREATE TABLE
products (...);

INSERT INTO
products
(name,price,...)
VALUES
('Mouse',299.99,
 ...);
```

No CREATE TABLE needed — MongoDB creates collections automatically on first insert

Insert Multiple:

```python
products.insert_many([{"name": "Keyboard", ...}, {"name": "Monitor",
...}])
```

MongoDB — Find / Query

Get all	`products.find()`	`SELECT * FROM products`
Filter	`products.find({"category": "Electronics"})`	`SELECT * FROM products` `WHERE category='Electronics'`
Comparison	`products.find({"price": {"$gt": 100}})`	`SELECT * FROM products` `WHERE price > 100`
Projection	`products.find({}, {"name": 1, "price": 1})`	`SELECT name, price` `FROM products`
Sort & Limit	`products.find().sort("price", -1).limit(5)`	`SELECT * FROM products` `ORDER BY price DESC LIMIT 5`

MongoDB operators: `$gt` (>) `$lt` (<) `$gte` (>=) `$lte` (<=) `$ne` (!=) `$in` ([...])

MongoDB — Update

update_one

```
products.update_one(
  {"name": "Wireless Mouse"},
  {"$set": {"price": 249.99, "stock": 45}}
)
```

```
UPDATE products
SET price=249.99,
    stock=45
WHERE name='Mouse'
```

update_many

```
products.update_many(
  {"category": "Electronics"},
  {"$inc": {"price": -50}}  # decrease price by 50
```

Common Update Operators

$set	Set field value	*SET col = value*
$inc	Increment by amount	*SET col = col + n*
$unset	Remove a field	*ALTER TABLE DROP (no equiv)*
$push	Add to array	*No SQL equivalent*

MongoDB — Delete

delete_one

```
products.delete_one({"name": "Wireless Mouse"})
```

```
DELETE FROM products
WHERE name='Mouse'
```

delete_many

```
products.delete_many({"stock": 0})
```

```
DELETE FROM products
WHERE stock = 0
```

Other Useful Operations

`products.count_documents({"category": "Electronics"})`	`SELECT COUNT(*) ... WHERE ...`	Count matching
`products.drop()`	`DROP TABLE products`	Delete collection
`products.distinct("category")`	`SELECT DISTINCT category ...`	Unique values

Quick Recap

Concept	SQL	MongoDB
Storage unit	`Table`	`Collection`
Single record	`Row`	`Document`
Field definition	`Column (fixed)`	`Key (flexible)`
Unique identifier	`PRIMARY KEY`	`_id (auto)`
Create	`INSERT INTO`	`insert_one()`
Read	`SELECT ... WHERE`	`find({filter})`
Update	`UPDATE ... SET`	`update_one($set)`
Delete	`DELETE FROM`	`delete_one()`

Now let's practice! You have 1 hour for 2 questions.